

WSI – Raport z ćwiczenia 6

Uczenie się ze wzmocnieniem

Karol Zalewski 319128

1. Wprowadzenie

Celem ćwiczenia była implementacja algorytmu Q-learning z epizodami oraz ϵ -zachłanną strategią wyboru akcji. Następnie należało wytrenować agenta rozwiązującego problem Four Rooms, a ostatecznie przeanalizować wpływ parametrów beta, gamma oraz epsilon na efektywność nauczania.

2. Założenia projektu

Środowisko dla problemu Four Rooms zostało wzięte z pakietu minigrid. Stan był reprezentowany przez trzy wartości (współrzędna x, współrzędna y oraz kierunek). Pozwoliło to na zmniejszenie rozmiaru tabeli Q i przyspieszenie algorytmu.

Agent i cel znajdowały się na stałych pozycjach – odpowiednio (7,7) oraz (2,2). Miało to na celu eliminację losowości i skupienie się na analizie parametrów uczenia.

Za dotarcie do celu była przyznawana standardowa nagroda ze środowiska, co zostało uzupełnione o małą karę za każdy krok w celu zmotywowania agenta do szukania najkrótszej drogi (gdyż im dłużej agent spędzał poruszając się po pokojach, tym większa była kara).

Strategia wyboru akcji: Zastosowano strategię ϵ -zachłanną, która balansuje między eksploracją (szukaniem nowych ścieżek) a wykorzystaniem już zdobytej wiedzy.

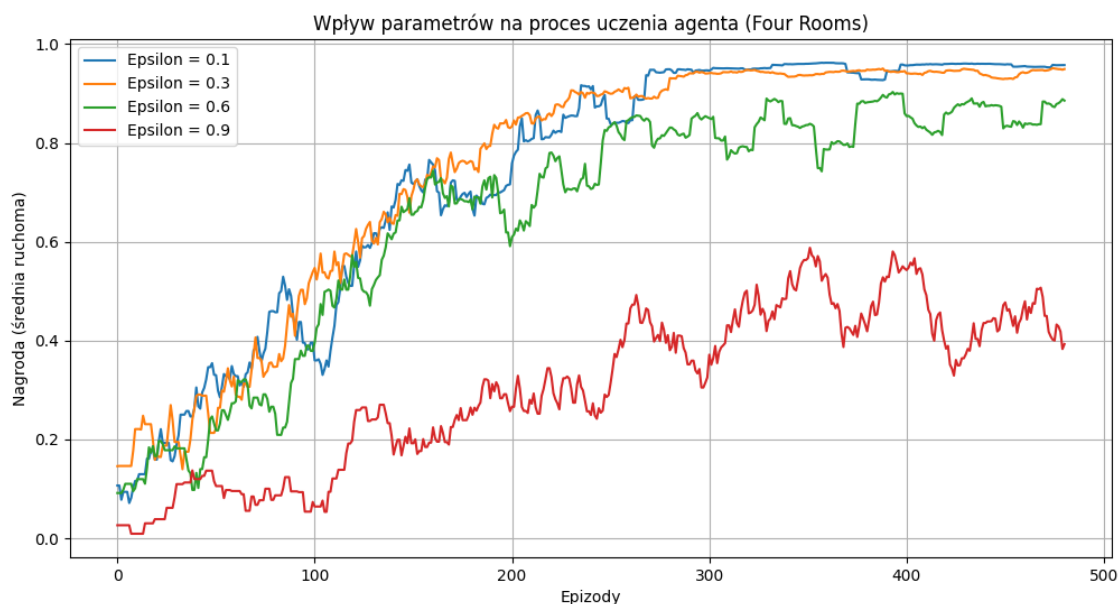
3. Prezentacja wyników

W celu umożliwienia algorytmowi znalezienia optymalnego rozwiązania, a jednocześnie nie wydłużaniu zbytnio czasu działania programu, zostały przyjęte wartości zmiennych:

- `max_steps` = 300 – jeden epizod miał trwać maksymalnie 300 kroków (aby agent nie błądził zbyt długo, ale żeby miał możliwość przypadkowego znalezienia rozwiązania w fazie początkowej).
- `episodes` = 500 - każdy proces treningowy trwał 500 epizodów, co pozwalało zaobserwować na wykresach różnice w prędkości uczenia, a także moment stabilizacji dla lepszych zestawów parametrów.

Po wstępnych eksperymentach przyjęte zostały parametry: $\beta = 0.1$, $\gamma = 0.9$ oraz $\epsilon = 0.1$. Następnie zostały przeprowadzone eksperymenty porównujące między sobą różne wartości jednego parametru przy stałych wartościach pozostałych.

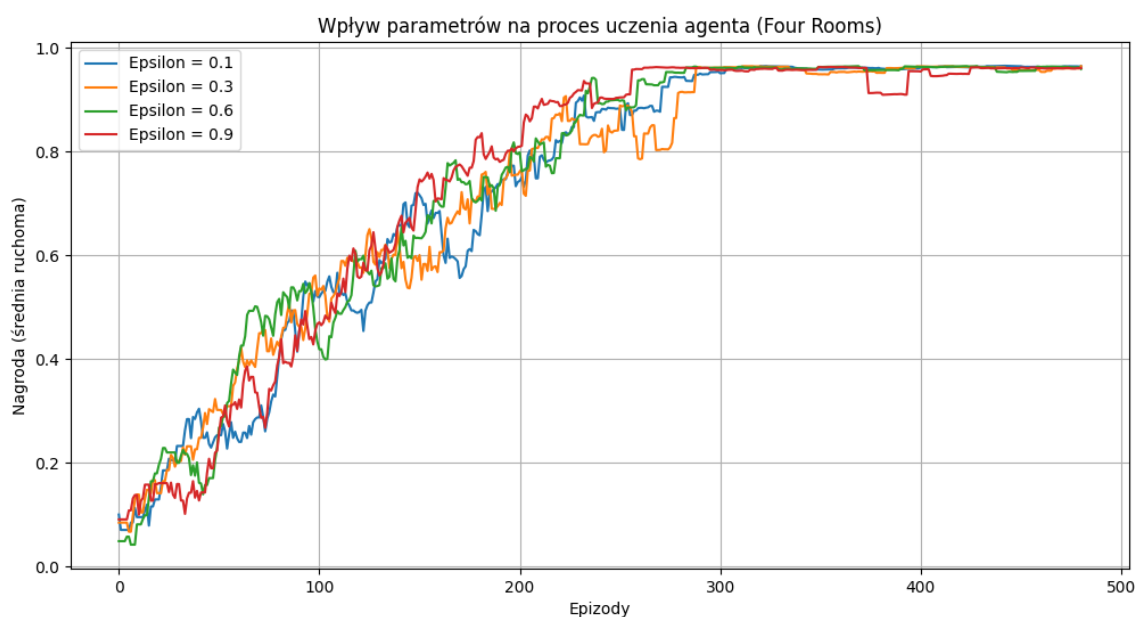
1. Test epsilon (0.1, 0.3, 0.6, 0.9), beta = 0.1, gamma = 0.9



Jak widać, zwiększanie wartości epsilon zmniejszało efektywność algorytmu. Agent zbyt często podejmował losowe akcje, zamiast takich, które były już zapamiętane jako optymalne.

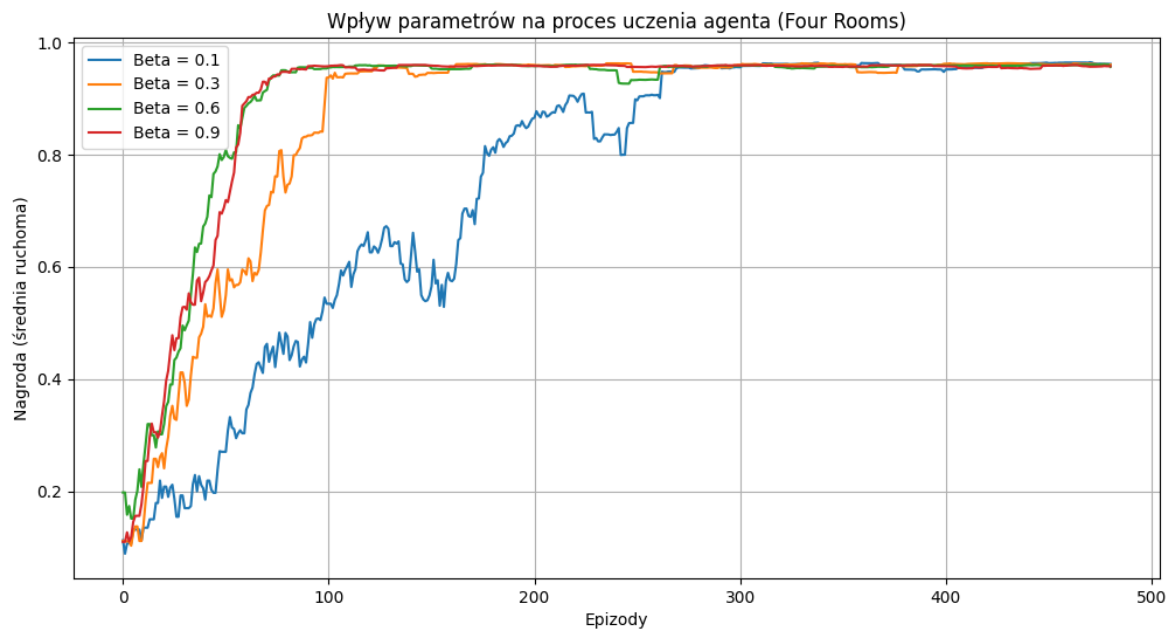
Zastosowanie zanikania parametru epsilon (mnożenie przez 0.99 co każdy epizod) spowodowało znaczną stabilizację wyników:

2. Test epsilon (0.1, 0.3, 0.6, 0.9), beta = 0.1, gamma = 0.9, zanikanie epsilon



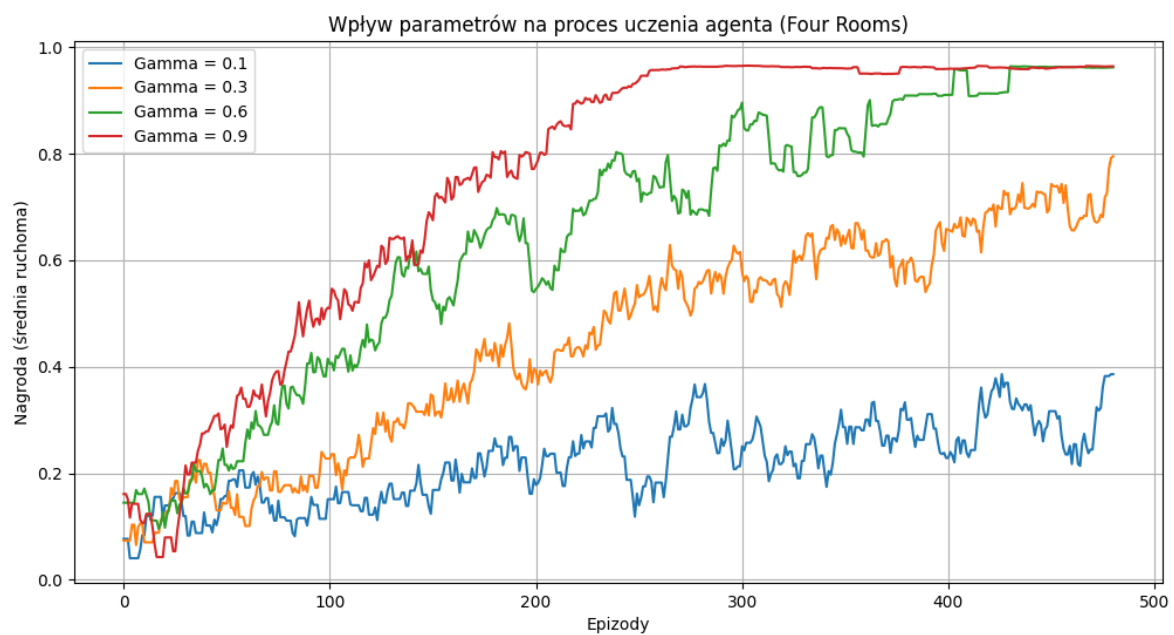
Następnie zostały przetestowane pozostałe dwa parametry (beta i gamma). W dalszych testach zanikanie epsilon było również włączone.

3. Test beta (0.1, 0.3, 0.6, 0.9), gamma = 0.9, epsilon = 0.1



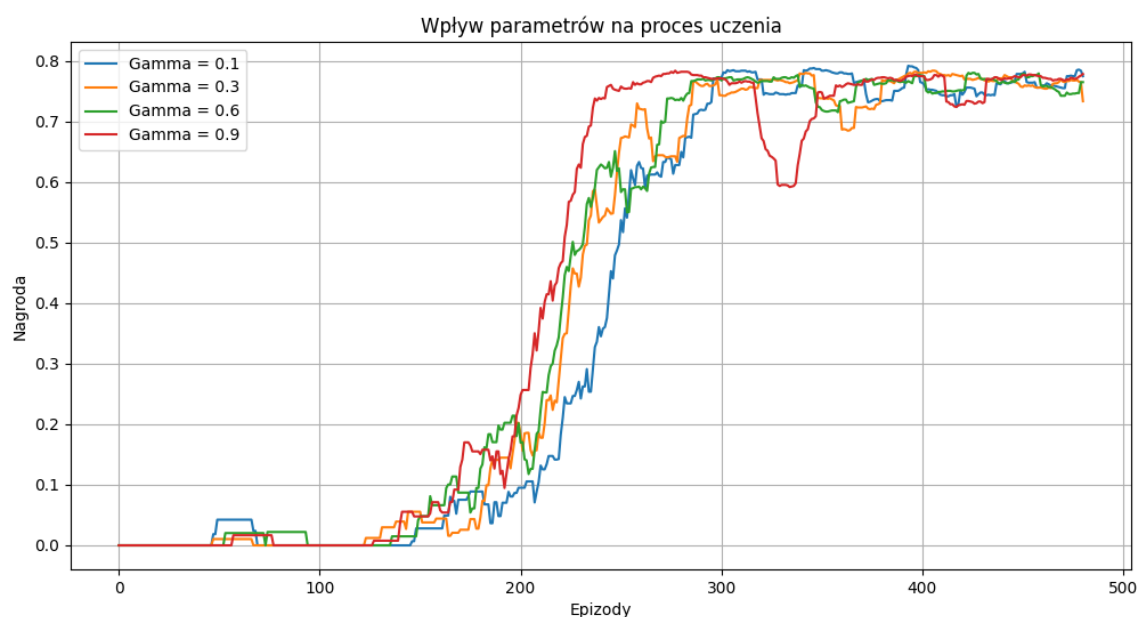
Zwiększanie parametru beta powodowało szybsze osiągnięcie przez algorytm stabilizacji.

4. Test gamma (0.1, 0.3, 0.6, 0.9), beta = 0.1, epsilon = 0.1



Podobnie jak w przypadku parametru beta, zwiększanie parametru gamma przyspieszało osiągnięcie wysokiej nagrody.

5. Test dla max_steps = 50 (zmienne gamma, beta = 0.9, epsilon = 0.1)



4. Wnioski

Eksperymenty potwierdziły, że w zadaniach wymagających dużej ilości kroków do osiągnięcia celu, wysoka wartość gamma (bliska 1.0) jest niezbędna. Przy niskich wartościach agent staje się „krótkowzroczny” i nie potrafi powiązać wczesnych ruchów (np. wyjścia z pierwszego pokoju) z odległą nagrodą końcową.

Wysoka wartość beta pozwalała na szybszy wzrost nagrody w początkowych epizodach. Mimo szybszego nadpisywania starej wiedzy, wykresy nie zdawały się być mniej stabilne (poszarpane), niż gdy używane były mniejsze wartości beta. Niższe wartości beta pozwalały z kolei na dostrzeżenie większych różnic przy zmianie innych parametrów (gamma lub epsilon).

Utrzymanie stałego, wysokiego poziomu epsilon uniemożliwiało agentowi osiągnięcie maksymalnej wydajności, ponieważ nawet po wyuczeniu optymalnej ścieżki, agent zbyt często podejmował akcje losowe. Zastosowanie mechanizmu zanikania epsilon pozwoliło na bardziej intensywną eksplorację na początku, a następnie kierowanie się zapamiętanymi wynikami, co skutkowało stabilizacją nagrody na wysokim poziomie niezależnie od wartości parametru.

Wprowadzenie niewielkiej ujemnej kary za każdy wykonany krok (-0.01) skutecznie wyeliminowało problem stania w miejscu. Agent był karany coraz bardziej, im dłużej spędzał na błąkanii się po labiryncie. Umożliwiło to osiągnięcie wyższej nagrody i ewentualną stabilizację algorytmu.